
SUPPLEMENTARY MATERIAL FOR A CASE STUDY OF STAGED METRIC-GATED GRPO FOR VISUAL NUMERIC REASONING

Kunal Ghosh

Purdue University
West Lafayette, Indiana, USA
ghosh178@purdue.edu
kg.aero@gmail.com

1 PURPOSE OF THE SUPPLEMENT

This supplement expands the main paper in four directions:

1. the exact reward and evaluation formulas used by the method,
2. the controller rules and checkpoint-selection logic,
3. the low-memory runtime profile used by the reported runs, and
4. the continuation semantics used for same-phase resume versus cross-phase warm start.

All statements below are grounded in the following implementation modules:

- `staged_rl/config.py`
- `staged_rl/rewarding.py`
- `staged_rl/evaluation.py`
- `staged_rl/controller.py`
- `staged_rl/checkpointing.py`
- `staged_rl/trainer_runtime.py`

2 BACKGROUND: GRPO VERSUS THE CONTRIBUTION OF THIS PAPER

Our implementation uses TRL’s `GRPOTrainer` as its optimization backend. The methodological contribution analyzed in the main paper is the fine-tuning pipeline built around that backend rather than a new GRPO objective:

- staged curriculum construction over filtered MathVista subsets,
- a structured response contract,
- metric-gated reward reweighting after checkpoint evaluation, and
- alias-based checkpoint selection for continuation.

GRPO itself provides the policy-optimization backbone: the trainer samples multiple completions for a prompt, evaluates them under a reward function, and updates the policy from relative within-group performance rather than from PPO-style clipped ratios. The contribution in this work is the fine-tuning pipeline built around that optimizer: filtered curricula, metric-gated reward adaptation, checkpoint-aware continuation, and auditable transfer semantics. This distinction is important because the optimization objective remains standard TRL GRPO, while the staged controller and continuation logic determine how the pipeline prioritizes structure, correctness, and checkpoint selection over time.

3 FULL REWARD SPECIFICATION

Let c denote a completion, g the gold answer, $m \in \{\text{num}, \text{mc}\}$ the answer mode, and L the phase-specific completion budget. Here num denotes the numeric free-form path and mc the multi-choice path. The total per-sample reward is

$$R(c, g, m) = w_f R_{\text{format}} + w_p R_{\text{parseable}} + w_h R_{\text{finished}} + w_c R_{\text{correctness}} + w_b R_{\text{brevity}} + w_t R_{\text{tolerance}}. \quad (1)$$

Tag and parsing predicates. The implementation uses:

$$\text{ReasoningTag}(c) = \mathbf{1}[\text{exactly one reasoning block exists}], \quad (2)$$

$$\text{SolutionTag}(c) = \mathbf{1}[\text{exactly one solution block exists}], \quad (3)$$

$$\text{Malformed}(c, m) = \begin{cases} \mathbf{1}[\text{numeric extraction or parsing fails}], & m = \text{num}, \\ \mathbf{1}[\text{option extraction fails}], & m = \text{mc}, \end{cases} \quad (4)$$

$$\text{Finished}(c, m) = \begin{cases} 1, & \text{ReasoningTag}(c) = 1 \wedge \text{SolutionTag}(c) = 1 \\ & \wedge \text{Malformed}(c, m) = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

Reward components. The six reward components are:

$$R_{\text{format}}(c, m) = \text{ReasoningTag}(c) + \text{SolutionTag}(c) - 0.5 \text{Malformed}(c, m), \quad (6)$$

$$R_{\text{parseable}}(c, m) = \begin{cases} \mathbf{1}[\text{numeric parsing succeeds}], & m = \text{num}, \\ \mathbf{1}[\text{Malformed}(c, m) = 0], & m = \text{mc}, \end{cases} \quad (7)$$

$$R_{\text{finished}}(c, m) = \begin{cases} 1, & \text{Finished}(c, m) = 1 \wedge \text{tok}(c) < L, \\ -1, & \text{tok}(c) \geq L \wedge \text{Finished}(c, m) = 0, \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

$$R_{\text{correctness}}(c, g, m) = \mathbf{1}[\text{exact match}], \quad (9)$$

$$R_{\text{brevity}}(c) = \begin{cases} -1, & \text{tok}(c) \geq L, \\ 1, & \text{tok}(c)/L \leq 0.55, \\ 0.5, & 0.55 < \text{tok}(c)/L \leq 0.75, \\ 0, & 0.75 < \text{tok}(c)/L \leq 0.90, \\ -0.5, & 0.90 < \text{tok}(c)/L < 1, \end{cases} \quad (10)$$

$$R_{\text{tolerance}}(c, g, m) = \begin{cases} 1, & \text{tolerance is enabled, exact match is false,} \\ & \text{and the tolerance check holds,} \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

Tolerance reward is active only in Phases C and D. Exact match suppresses tolerance reward, so the tolerance component acts strictly as a denser near-miss signal rather than an additional bonus for already-correct answers.

Phase-initialized weights. The configured initial weights are manual curriculum defaults rather than values learned by a meta-optimizer:

Phase	Format	Parseable	Finished	Correctness	Brevity	Tolerance
A	1.0	1.0	1.5	2.0	0.25	0.0
B	0.75	0.75	1.0	4.0	0.20	0.0
C	0.50	0.50	0.75	5.0	0.20	1.0
D	0.50	0.50	0.75	5.0	0.30	1.0

4 EVALUATION METRICS AND CHECKPOINT SCORES

For each prompt i , the evaluator samples one or more completions $c_{i,j}$. It computes:

$$\text{EM} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{first sampled completion matches exactly}], \quad (12)$$

$$\text{TolAcc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{first sampled completion is correct under tolerance}], \quad (13)$$

$$\text{ParseRate} = \frac{1}{M} \sum_{i,j} \mathbf{1}[\text{sample } (i, j) \text{ is parseable}], \quad (14)$$

$$\text{MalRate} = \frac{1}{M} \sum_{i,j} \mathbf{1}[\text{sample } (i, j) \text{ is malformed}], \quad (15)$$

$$\text{TruncRate} = \frac{1}{M} \sum_{i,j} \mathbf{1}[\text{sample } (i, j) \text{ is truncated}]. \quad (16)$$

It also computes tag-compliance rates, completion-token averages, repetition rate, sampled-answer diversity, and mean logged reward components.

The checkpoint registry then defines three scores:

$$\begin{aligned} S_{\text{struct}} = & 0.35 \text{ ParseRate} + 0.20 \text{ SolutionTag} \\ & + 0.20 \text{ ReasoningTag} - 0.15 \text{ MalRate} \\ & - 0.10 \text{ TruncRate}, \end{aligned} \quad (17)$$

$$S_{\text{corr}} = 0.70 \text{ EM} + 0.20 \text{ TolAcc} + 0.10 \text{ ParseRate}, \quad (18)$$

$$\begin{aligned} S_{\text{comp}} = & 0.45 \text{ EM} + 0.15 \text{ TolAcc} + 0.15 \text{ ParseRate} \\ & + 0.10 \text{ SolutionTag} + 0.05 \text{ ReasoningTag} \\ & - 0.05 \text{ MalRate} - 0.05 \text{ TruncRate}. \end{aligned} \quad (19)$$

These scores back the aliases `best_structure`, `best_correctness`, and `best_composite`. The latest alias is purely chronological.

5 CONTROLLER RULES AND THRESHOLDS

The controller audit covers 26 evaluated checkpoints. Across those checkpoints:

- parseability guard fires 2 times,
- formatting guard fires 2 times,
- finish guard fires 2 times,
- correctness escalation fires 12 times, and
- 6 correctness escalations occur when exact match has regressed rather than merely plateaued.

This behavior is consistent with the code: once the system judges structure to be stable, it reacts to weak exact-match progress by raising correctness pressure even if the immediate trend is negative.

6 RUNTIME PROFILE AND RESOURCE-CONSTRAINED OPERATION

The successful runs share the same low-memory profile:

- max sequence length reduced from 16384 to 1280,
- image size reduced from 512 to 336,
- LoRA rank reduced from 16 to 8,

Table 1: Controller rules implemented in `staged_rl/controller.py`.

Rule	Trigger	Action
Parseable guard	parseable answer rate < 0.85	Raise parseable reward to at least 0.75
Format guard	solution tag < 0.90 or	
reasoning tag < 0.88		
or		
malformed rate > 0.10	Raise format reward to at least 0.75	
Finish guard	truncation rate > 0.15 or	
average completion	Add 0.25 to finished reward	
tokens exceed $0.85L$		
Stable structure test	parseable ≥ 0.92 ,	
solution ≥ 0.95 ,		
reasoning ≥ 0.93 ,		
malformed ≤ 0.05 ,		
truncation ≤ 0.08	Enable plateau-based correctness escalation once enough history exists	
Correctness	Stable structure holds over a	Add 0.50 to correctness reward
escalation	two-checkpoint window and exact-match gain is below 0.02	

- sampled generations reduced from 4 to 2,
- max prompt length reduced from 1024 to 320,
- max completion length reduced from 256 to 64 in the default Kaggle profile,
- max completion length increased to 96 only for Phase D,
- evaluation reduced to one sample per prompt and at most two examples per subset.

We interpret these changes as runtime stabilizers rather than task simplifiers. They reduce per-step VRAM pressure and evaluation overhead, but they also reduce reasoning budget, visual detail, and generation diversity.

Runtime configuration in code form. The same runtime profile can be read directly in the style of the code paths that consume these settings:

```
# Model load / LoRA adapter profile
FastVisionModel.from_pretrained(
    model_name="unsloth/Qwen2.5-VL-7B-Instruct",
    max_seq_length=1280,
    load_in_4bit=True,
    fast_inference=True,
    gpu_memory_utilization=0.65,
    max_lora_rank=8,
    compilation_config={"level": 3, "cudagraph_mode": "PIECEWISE"},
)

FastVisionModel.get_peft_model(
    model,
    r=8,
    lora_alpha=8,
    finetune_vision_layers=False,
    finetune_language_layers=True,
    finetune_attention_modules=True,
    finetune_mlp_modules=True,
)
```

```
# Trainer / evaluation profile
GRPOConfig(
    gradient_accumulation_steps=4,
```

```

        num_generations=2,
        max_prompt_length=320,
        max_completion_length=64, # Phase D override: 96
        importance_sampling_level="sequence",
        loss_type="dr_grpo",
    )

    SamplingParams(
        temperature=1.0,
        top_k=50,
        max_tokens=64, # Phase D override: 96
        n=1,
    )

    EvalConfig(
        num_samples_per_prompt=1,
        max_eval_examples_per_subset=2,
    )

```

7 TRANSFER AND CONTINUATION SEMANTICS

The code distinguishes three different mechanisms that can be conflated in informal descriptions:

1. **Cross-notebook transfer.** A checkpoint directory is physically moved through a Kaggle dataset attachment or another external bundle.
2. **Warm start.** A fresh base model is loaded, a fresh LoRA adapter is attached, and adapter weights are copied from the selected checkpoint into that new adapter.
3. **Same-phase resume.** Full trainer state is restored from a checkpoint, including optimizer state, scheduler state, trainer progress, and controller state.

This distinction is important for the claims in the paper. Cross-phase continuation often uses alias-based warm start rather than exact trainer resume. Same-phase restarts can use exact trainer resume if the selected checkpoint belongs to the current phase.

8 ADDITIONAL NOTES ON SCOPE

The implementation contains scaffolding for a multi-choice branch and an optional robustness stage, but neither path provides the main evidence used in the paper. Similarly, the runner logs when multiple GPUs are visible but does not automatically switch to DDP or tensor parallelism. The strongest claims are therefore:

- about staged GRPO fine-tuning pipeline dynamics for numeric visual reasoning,
- about metric-gated control under non-monotonic checkpoint trajectories, and
- about making a single-GPU GRPO fine-tuning pipeline more auditable and reproducible.